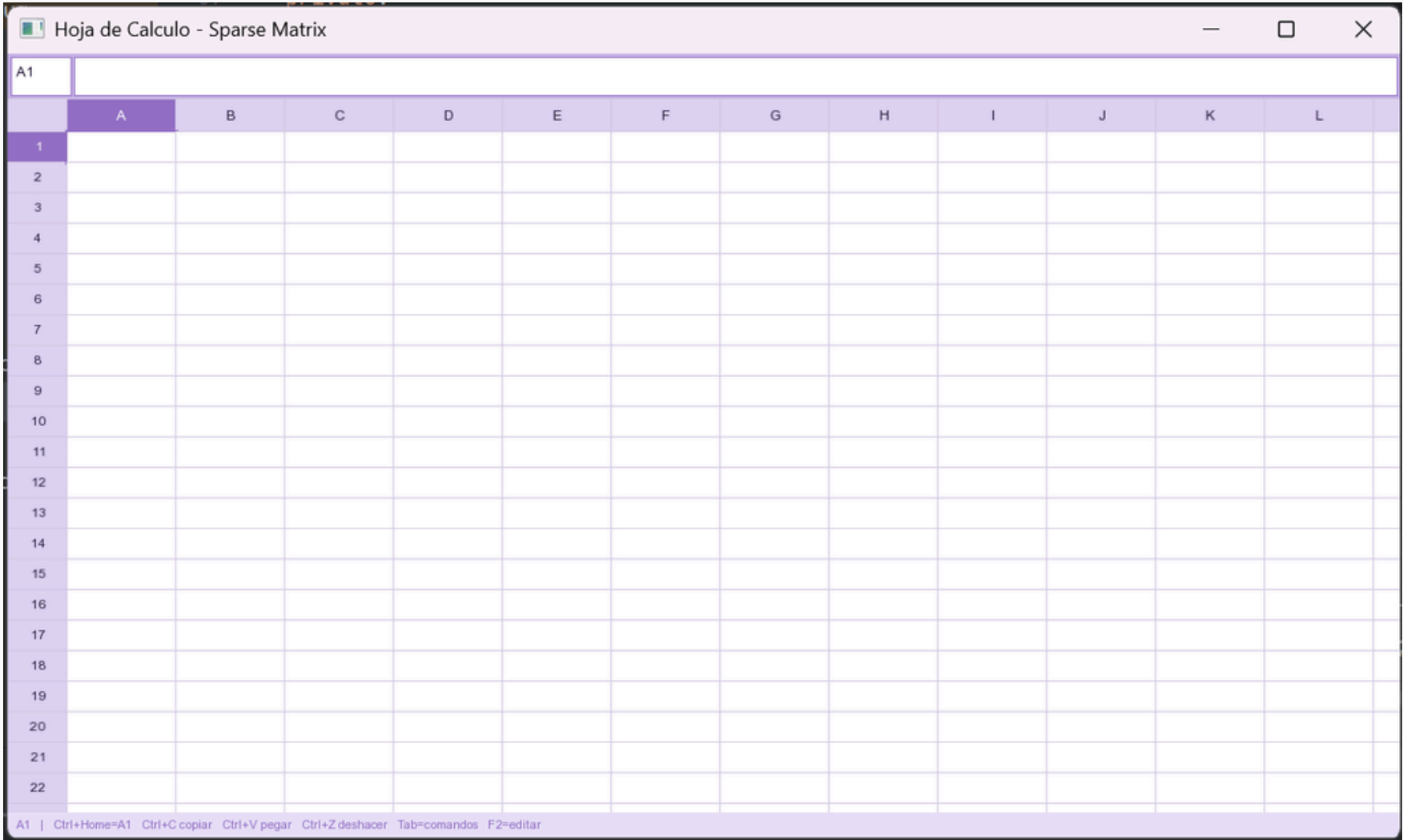


Hoja de Cálculo con Matrices Dispersas



Motivacion

En una hoja de calculo real, la gran mayoria de celdas estan vacias.

- Una matriz densa $T[N][M]$ reserva memoria para todas las celdas posibles, incluso vacias.
- No se puede crecer mas alla del tamaño fijo declarado.
- Una matriz dispersa permite crecimiento dinámico y almacena únicamente celdas con datos reales.

Comparacion cuantitativa

Aspecto	Matriz densa	Matriz dispersa
Memoria reservada	$10^6 \cdot 8 \text{ B} \approx \mathbf{8 \text{ MB}}$	$500 \cdot 56 \text{ B} \approx \mathbf{28 \text{ KB}}$
Memoria desperdiciada	99.95 %	0 %
Acceso a celda (i, j)	$O(1)$ aleatorio	$O(R + k)$
Insertar nueva celda	$O(1)$ (si cabe)	$O(R + C + k + m)$
Eliminar celda	$O(1)$ marcado	$O(R + C + k)$ con punteros
Crecimiento dinamico	no	si, ilimitado

Donde: R, C = filas/columnas activas; k, m = nodos en la fila/columna; K = nodos totales.

En el caso de usar hoja de 1,000×1,000 con solo 500 celdas ocupadas (0.05%)

- En una hoja de cálculo real, la mayoría de celdas suelen estar vacías, por lo que reservar memoria para todas ellas sería ineficiente.

ESTRUCTURA DEL NODO

Clase Node<T>

```
template <typename T>
struct Node {
    T          data;           // valor numerico o texto (variant)
    int        row, col;       // coordenadas en la hoja
    Node<T>*   nextInRow;      // siguiente celda en la misma fila
    Node<T>*   prevInRow;      // anterior celda en la misma fila
    Node<T>*   nextInCol;      // siguiente celda en la misma columna
    Node<T>*   prevInCol;      // anterior celda en la misma columna

    Node(T d, int r, int c) : data(d), row(r), col(c),
        nextInRow(nullptr), prevInRow(nullptr),
        nextInCol(nullptr), prevInCol(nullptr) {}
};
```

Doblemente enlazado en ambas direcciones $\Rightarrow$ eliminacion sin recorrer hacia atras.

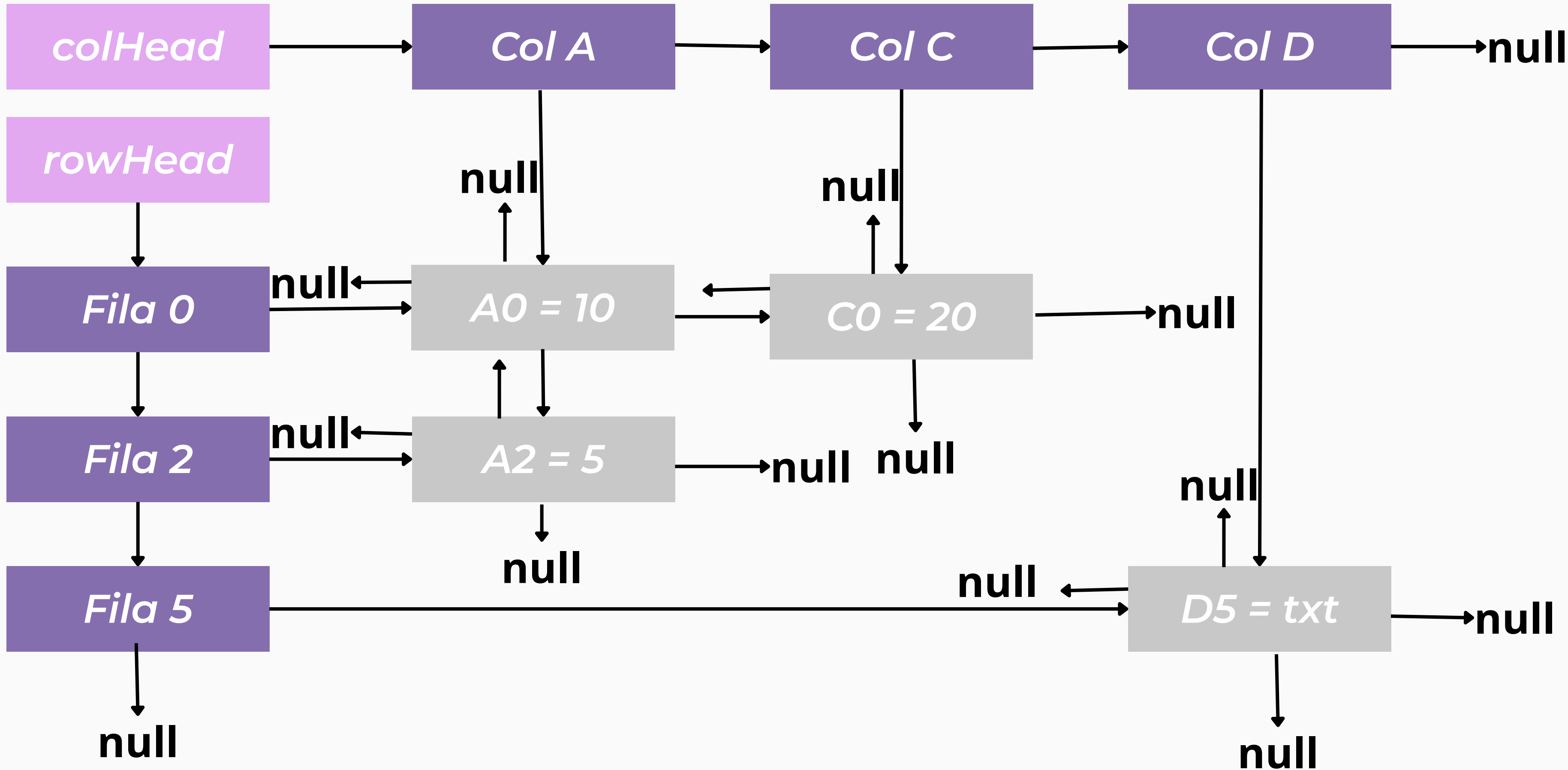
NODOS CABECERA

Clase HeadNode<T>

```
template<typename T>
struct HeadNode {
    int          index;      // numero de fila o columna
    HeadNode<T>* next;      // siguiente cabecera en la lista
    Node<T>*     first;      // primer nodo de esa fila/columna

    HeadNode(int idx) : index(idx), next(nullptr), first(nullptr) {}
};
```


DIAGRAMA: SPARSE MATRIX IMPLEMENTADA CON LISTA ENLAZADA CRUZADA



ANALISIS DE LA COMPLEJIDAD

Operacion	Complejidad
<code>set(i,j,v)</code>	$O(R + C + k + m)$
<code>get(i,j)</code>	$O(R + k)$
<code>update(i,j,v)</code>	$O(R + k)$
<code>remove(i,j)</code>	$O(R + C + k)$
<code>remove_row(i)</code>	$O(R + k \cdot C)$
<code>remove_col(j)</code>	$O(C + m \cdot R)$
<code>remove_range</code>	$O(R + k_{range} \cdot C)$
<code>sum_row / sum_col</code>	$O(R + k) / O(C + m)$
<code>sum_range</code>	$O(R + k_{range})$
<code>max/min/avg_range</code>	$O(R + k_{range})$

Donde: R, C = filas/columnas activas; k, m = nodos en la fila/columna; K = nodos totales.

INSERTAR O ACTUALIZAR UNA CELDA

set(i,j,value)

```
void set(int i, int j, T value) {
    HeadNode<T>* row = getOrCreateHead(rowHead, i);
    HeadNode<T>* column = getOrCreateHead(colHead, j);

    Node<T>* prevRow = nullptr;
    Node<T>* currRow = row->first;
    while (currRow && currRow->col < j) {           // mantener orden
        prevRow = currRow; currRow = currRow->nextInRow;
    }
    if (currRow && currRow->col == j) {             // ya existe -> update
        currRow->data = value; return;
    }
    Node<T>* newNode = new Node<T>(value, i, j);
    newNode->nextInRow = currRow; newNode->prevInRow = prevRow;
    if (prevRow) prevRow->nextInRow = newNode; else row->first = newNode;
    if (currRow) currRow->prevInRow = newNode;
    // ... mismo procedimiento para la lista de columna ...
    if (i > maxRow) maxRow = i;
    if (j > maxCol) maxCol = j;
}
```


CONSULTAR VALOR DE CELDA

get(i,j)

```
T get(int i, int j) const {
    if (i < 0 || j < 0) throw std::out_of_range("Indexes out of bounds");
    if (i > maxRow || j > maxCol) return T{};           // mas alla de los datos

    HeadNode<T>* temp1 = rowHead;
    while (temp1 && temp1->index < i) temp1 = temp1->next;
    if (!temp1 || temp1->index != i) return T{};         // fila no existe

    Node<T>* temp2 = temp1->first;
    while (temp2 && temp2->col < j) temp2 = temp2->nextInRow;
    if (!temp2 || temp2->col != j) return T{};          // columna no esta

    return temp2->data;
}
```

ACTUALIZAR VALOR DE CELDA

update(i,j,value)

```
bool update(int i, int j, T value){
    if (i < 0 || j < 0 || i > maxRow || j > maxCol) { // Índices negativos o fuera del tamaño de la matriz
        return false;
    }

    HeadNode<T>* temp1 = rowHead;
    while (temp1 != nullptr && temp1->index < i) { // Recorrer índices de fila, O(i)
        temp1 = temp1->next;
    }

    if (temp1 == nullptr || temp1->index != i) {
        return false;
    }

    Node<T>* temp2 = temp1->first;
    while (temp2 != nullptr && temp2->col < j) { // Recorrer celdas en la fila, O(j)
        temp2 = temp2->nextInRow;
    }

    if (temp2 == nullptr || temp2->col != j) {
        return false;
    }

    temp2->data = value; // Actualización de celda
    return true;
}
```

ELIMINAR CELDA

remove(i,j)

```
// Eliminar celda
bool remove(int i, int j) {
    if (i < 0 || j < 0 || i > maxRow || j > maxCol)
        throw std::out_of_range("Indexes out of bounds");

    HeadNode<T>* row = rowHead;

    while (row && row->index < i) { //O(i)
        row = row->next;
    }

    if (!row || row->index != i) return false;

    Node<T>* curr = row->first;
    while (curr && curr->col < j) { // O(j)
        curr = curr->nextInRow;
    }

    if (!curr || curr->col != j) return false;

    HeadNode<T>* col = colHead;

    while (col && col->index < j) { // O(j)
        col = col->next;
    }

    if (!col || col->index != j) return false;

    //Desconexión y eliminación en O(1)
    if (curr->prevInRow)
        curr->prevInRow->nextInRow = curr->nextInRow; // Si existe anterior, lo conectamos con el siguiente
    else
        row->first = curr->nextInRow; // Sino, conectamos la cabecera con el siguiente

    if (curr->nextInRow)
        curr->nextInRow->prevInRow = curr->prevInRow; //Si existe siguiente, lo conectamos con el anterior
}
```

ELIMINAR CELDA

remove(i,j)

```
//Hacemos lo mismo para columnas
if (curr->prevInCol)
    curr->prevInCol->nextInCol = curr->nextInCol;
else
    col->first = curr->nextInCol;

if (curr->nextInCol)
    curr->nextInCol->prevInCol = curr->prevInCol;

delete curr;

return true;
}
```

ELIMINAR FILA

remove_row(i)

```
// Eliminar fila
bool remove_row(int i) {
    if (i < 0 || i > maxRow)
        throw std::out_of_range("Index out of bounds");
    HeadNode<T>* rowPrev = nullptr;
    HeadNode<T>* row = rowHead;
    while (row && row->index < i) {
        rowPrev = row;
        row = row->next;
    }
    if (!row || row->index != i)
        return false;
    Node<T>* curr = row->first;
    while (curr) {
        Node<T>* next = curr->nextInRow;
        HeadNode<T>* col = colHead;
        while (col && col->index < curr->col)
            col = col->next;

        if (col) {
            if (curr->prevInCol)
                curr->prevInCol->nextInCol = curr->nextInCol;
            else
                col->first = curr->nextInCol;
            if (curr->nextInCol)
                curr->nextInCol->prevInCol = curr->prevInCol;
        }
        delete curr;
        curr = next;
    }
    if (rowPrev)
        rowPrev->next = row->next;
    else
        rowHead = row->next;
    delete row;
    return true;
}
```


ELIMINAR COLUMNA

remove_col(j)

```
// Eliminar columna
bool remove_col(int j) {
    if (j < 0 || j > maxCol)
        throw std::out_of_range("Index out of bounds");
    HeadNode<T>* colPrev = nullptr;
    HeadNode<T>* col = colHead;
    while (col && col->index < j) {
        colPrev = col;
        col = col->next;
    }
    if (!col || col->index != j)
        return false;
```

```
Node<T>* curr = col->first;
while (curr) {
    Node<T>* next = curr->nextInCol;
    HeadNode<T>* row = rowHead;
    while (row && row->index < curr->row)
        row = row->next;
    if (row) {
        if (curr->prevInRow)
            curr->prevInRow->nextInRow = curr->nextInRow;
        else
            row->first = curr->nextInRow;
        if (curr->nextInRow)
            curr->nextInRow->prevInRow = curr->prevInRow;
    }
    delete curr;
    curr = next;
}
if (colPrev)
    colPrev->next = col->next;
else
    colHead = col->next;
delete col;
return true;
}
```

```
// Eliminar rango de celdas
bool remove_range(int i1, int j1, int i2, int j2) {
    if (i1 < 0 || j1 < 0 || i1 > i2 || j1 > j2)
        throw std::out_of_range("Indexes out of bounds");
    // Ajustar al límite real de datos – no hay nada que borrar más allá
    i2 = std::min(i2, maxRow);
    j2 = std::min(j2, maxCol);
    if (i2 < i1 || j2 < j1) return true;

```

```
    HeadNode<T>* row = rowHead;

```

```
    while (row && row->index < i1)
        row = row->next;

```

```
    while (row && row->index <= i2) {

```

```
        Node<T>* curr = row->first;

```

```
        while (curr) {

```

```
            Node<T>* next = curr->nextInRow;

```

```
            if (curr->col >= j1 && curr->col <= j2) {

```

```
                if (curr->prevInRow)

```

```
                    curr->prevInRow->nextInRow = curr->nextInRow;

```

```
                else

```

```
                    row->first = curr->nextInRow;

```

```
                if (curr->nextInRow)

```

```
                    curr->nextInRow->prevInRow = curr->prevInRow;

```

```
                if (curr->prevInCol)

```

```
                    curr->prevInCol->nextInCol = curr->nextInCol;

```

ELIMINAR RANGO DE CELDAS

remove_range(i1, i2, j1, jj2)

```
        else {
            HeadNode<T>* col = colHead;
            while (col && col->index < curr->col)
                col = col->next;
            if (col)
                col->first = curr->nextInCol;
        }
        if (curr->nextInCol)
            curr->nextInCol->prevInCol = curr->prevInCol;
        delete curr;
    }
    curr = next;
    row = row->next;
}

return true;
}
```


SUMA ELEMENTOS DE UNA FILA

sum_row(i)

```
// Suma de todos los elementos en una fila
double sum_row(int i) {
    if (i < 0 || i > maxRow)
        throw std::out_of_range("Index out of bounds");

    HeadNode<T>* temp1 = rowHead;
    while (temp1 != nullptr && temp1->index < i)
        temp1 = temp1->next;

    if (temp1 == nullptr || temp1->index != i)
        return 0.0;

    Node<T>* temp2 = temp1->first;
    double sumatoria = 0.0;
    while (temp2 != nullptr) {
        sumatoria += cellToDouble(temp2->data);
        temp2 = temp2->nextInRow;
    }
    return sumatoria;
}
```


SUMA ELEMENTOS DE UNA COLUMNA

sum_col(j)

```
// Suma de todos los elementos en una columna
double sum_col(int j) {
    if (j < 0 || j > maxCol)
        throw std::out_of_range("Index out of bounds");

    HeadNode<T>* temp1 = colHead;
    while (temp1 != nullptr && temp1->index < j)
        temp1 = temp1->next;

    if (temp1 == nullptr || temp1->index != j)
        return 0.0;

    Node<T>* temp2 = temp1->first;
    double sumatoria = 0.0;
    while (temp2 != nullptr) {
        sumatoria += cellToDouble(temp2->data);
        temp2 = temp2->nextInCol;
    }
    return sumatoria;
}
```

SUMA ELEMENTOS EN UN RANGO DE CELDA

sum_range(i1, i2, j1, j2)

```
// Suma de valores en un rango de celdas
double sum_range(int i1, int j1, int i2, int j2) {
    if (i1 < 0 || j1 < 0 || i2 > maxRow || j2 > maxCol || i1 > i2 || j1 > j2)
        throw std::out_of_range("Indexes out of bounds");

    double sumatoria = 0.0;
    HeadNode<T>* row = rowHead;
    while (row != nullptr && row->index < i1)
        row = row->next;

    while (row != nullptr && row->index <= i2) {
        Node<T>* curr = row->first;
        while (curr != nullptr && curr->col < j1)
            curr = curr->nextInRow;
        while (curr != nullptr && curr->col <= j2) {
            sumatoria += cellToDouble(curr->data);
            curr = curr->nextInRow;
        }
        row = row->next;
    }
    return sumatoria;
}
```

MÁXIMO VALOR EN UN RANGO DE CELDA

max_range(i1, i2, j1, j2)

```
// Máximo valor en un rango de celdas
double max_range(int i1, int j1, int i2, int j2) {
    if (i1 < 0 || j1 < 0 || i2 > maxRow || j2 > maxCol || i1 > i2 || j1 > j2)
        throw std::out_of_range("Indexes out of bounds");

    bool found = false;
    double maxVal = 0.0;

    HeadNode<T>* row = rowHead;
    while (row && row->index < i1) row = row->next;

    while (row && row->index <= i2) {
        Node<T>* curr = row->first;
        while (curr) {
            if (curr->col >= j1 && curr->col <= j2) {
                double d = cellToDouble(curr->data);
                if (!found || d > maxVal) { maxVal = d; found = true; }
            }
            curr = curr->nextInRow;
        }
        row = row->next;
    }

    if (!found) throw std::runtime_error("No elements in range");
    return maxVal;
}
```

MÍNIMO VALOR EN UN RANGO DE CELDAS

min_range(i1, i2, j1, j2)

```
// Mínimo valor en un rango de celdas
double min_range(int i1, int j1, int i2, int j2) {
    if (i1 < 0 || j1 < 0 || i2 > maxRow || j2 > maxCol || i1 > i2 || j1 > j2)
        throw std::out_of_range("Indexes out of bounds");

    bool found = false;
    double minVal = 0.0;

    HeadNode<T>* row = rowHead;
    while (row && row->index < i1) row = row->next;

    while (row && row->index <= i2) {
        Node<T>* curr = row->first;
        while (curr) {
            if (curr->col >= j1 && curr->col <= j2) {
                double d = cellToDouble(curr->data);
                if (!found || d < minVal) { minVal = d; found = true; }
            }
            curr = curr->nextInRow;
        }
        row = row->next;
    }

    if (!found) throw std::runtime_error("No elements in range");
    return minVal;
}
```

PROMEDIO DE VALORES EN UN RANGO DE CELDAS

`avg_range(i1, i2, j1, j2)`

```
// Promedio de valores en un rango de celdas
double avg_range(int i1, int j1, int i2, int j2) {
    if (i1 < 0 || j1 < 0 || i2 > maxRow || j2 > maxCol || i1 > i2 || j1 > j2)
        throw std::out_of_range("Indexes out of bounds");

    double suma = 0.0;
    int count = 0;

    HeadNode<T>* row = rowHead;
    while (row && row->index < i1) row = row->next;

    while (row && row->index <= i2) {
        Node<T>* curr = row->first;
        while (curr) {
            if (curr->col >= j1 && curr->col <= j2) {
                suma += cellToDouble(curr->data);
                count++;
            }
            curr = curr->nextInRow;
        }
        row = row->next;
    }

    if (count == 0) return 0.0;
    return suma / count;
}
```


MOTOR DE FORMULAS

formula.cpp

```
// "B3" -> {2, 1}
std::pair<int,int> parseCell(const std::string& input);

// =SUMA(A1:B5), =MAX(A1:B5), =A1+B2, =A1, =42
double evalFormula(const std::string& formula, SparseMatrix<CellValue>& mat);

// Comandos de la barra (sin '='): SUMA, MAX, MIN, PROMEDIO,
// CONTAR, LISTAR, VER_FILA, VER_COL
std::string execCommand(const std::string& cmd, SparseMatrix<CellValue>& mat);
```

INTERFAZ GRAFICA

SFML

Caracteristicas

- Grilla dinamica
- Headers A,B,C... y 1,2,3... y barra de formulas.
- Seleccion por arrastre o Shift+flechas.
- Copiar/Pegar contra el portapapeles del sistema.
- Undo de hasta 30 niveles
- Barra de estado en tiempo real: SUMA, CONTAR, PROMEDIO del rango seleccionado.
- Errores estilo Excel: #!VALOR! en rojo.

Atajos de teclado

Atajo	Accion
Cmd/Ctrl + C	Copiar
Cmd/Ctrl + V	Pegar
Cmd/Ctrl + Z	Deshacer
Shift + flechas	Extender seleccion
F2	Editar celda
Tab	Abrir comandos
Delete	Borrar seleccion
Enter	Confirmar y bajar
Escape	Volver a A1